

Atividade 09, Comandos

PLANEJAMENTO

Plano de execução, Compiladores, UFPB, julho de 2026

Grupo: Daniel Victor, Tiago Trindade, Yann Fabber, Ralf Dwerich

1. OBJETIVO

Construir um compilador para a linguagem **Cmd**, a primeira linguagem Turing completa da disciplina. Em relação à atividade anterior, a linguagem ganha comando condicional, comando de repetição, comando de atribuição e operadores de comparação. Isso obriga a mexer nas quatro fases do compilador, com o maior impacto na geração de código, que passa a precisar de saltos e rótulos.

2. ESCOPO DERIVADO DO GUIA

Item	O que precisa existir	Prioridade
Estrutura do programa	Zero ou mais declarações, corpo entre chaves com zero ou mais comandos e uma expressão <code>return</code>	obrigatório
Comandos	<code>if</code> com <code>else</code> , <code>while</code> e atribuição a variável já declarada	obrigatório
Comparações	<code><</code> , <code>></code> e <code>==</code> em um nível de precedência abaixo da soma	obrigatório
Verdade e falsidade	Zero é falso, qualquer outro valor é verdadeiro, sem tipo booleano separado	obrigatório
Verificação semântica	Erro ao usar ou atribuir variável não declarada, dos dois lados da atribuição	obrigatório
Programas de teste	Cobrando condicional, repetição e atribuição, com documentação de execução	obrigatório
Extensões	Seção Variações do guia: novos operadores, booleanos, entrada, <code>if</code> sem <code>else</code>	desejável

3. ESTRATÉGIA TÉCNICA

- Implementação em Python 3 puro**, sem gerador de parser e sem dependência externa. Descida recursiva escrita à mão, uma função por não terminal, para o código ficar próximo da gramática do guia.
- Alvo x86_64 em sintaxe AT&T**, montado e linkado pelo gcc. Seguimos o modelo de tradução do guia instrução por instrução, sem otimização.
- Interpretador de referência** escrito junto com o compilador. Ele executa a mesma árvore sintática e serve de oráculo: todo teste roda nas duas vias e os resultados precisam bater. É a forma mais barata de encontrar bug de geração de código.
- Erros com fase, linha e coluna** desde o analisador léxico, para que os casos de teste de erro possam ser verificados de forma automática.

4. PLANO POR FASE

Fase	O que muda em relação à atividade anterior	Entregável
Léxica	Tokens <code>{</code> <code>}</code> , operadores de comparação e palavras chave. Resolver <code>=</code> contra <code>==</code> por lookahead de um caractere e reclassificar identificador em palavra chave por consulta a conjunto	<code>lexico.py</code>
Sintática	Reconhecer lista de comandos, blocos com chaves obrigatórias e um novo nível de precedência para comparações. Manter <code>return</code> fora da lista de comandos	<code>sintatico.py</code> , <code>ast.py</code>
Semântica	Verificar os dois lados da atribuição contra a tabela de símbolos, sem inserir nada nela	<code>semantico.py</code>
Geração	Comparações com <code>CMP</code> e <code>SETcc</code> , condicional e repetição com <code>JZ</code> e rótulos únicos gerados por contador global	<code>codigo.py</code>
Apoio	Interpretador de referência, interface de linha de comando e runner de testes	<code>interp.py</code> , <code>cmdc.py</code>

Risco principal identificado desde o início: rótulos duplicados em `if` e `while` aninhados. Mitigação decidida antes de escrever código: um único contador global no gerador, incrementado a cada construção, concatenado no nome de todos os rótulos daquela construção.

5. EXTENSÕES PLANEJADAS

Extensão	Decisão
Comparações <code><=</code> <code>>=</code> <code>!=</code>	Fazer. Custo baixo, reaproveita o nível de precedência e só adiciona três instruções <code>SETcc</code> .
Booleanos <code>and</code> <code>or</code> <code>not</code>	Fazer. Três novos níveis de precedência abaixo das comparações, com curto circuito por salto condicional.
<code>if</code> sem <code>else</code>	Fazer. Sem ambiguidade de dangling else, já que as chaves são obrigatórias.
Comando de entrada	Fazer. <code>read v;</code> com <code>scanf</code> , escrevendo direto no símbolo da variável.
Atribuição criando variável nova	Não fazer. O guia avisa que complica a geração e é justamente a verificação semântica pedida na atividade. Fica registrado como possível trabalho futuro.

6. ESTRATÉGIA DE TESTES

- Cada programa de teste declara o resultado esperado em um comentário no topo do próprio arquivo, lido pelo runner.
- Todo programa válido roda duas vezes, no interpretador e no binário nativo, e os dois valores são comparados com o esperado. Divergência entre as duas vias aponta bug de geração de código.
- Os quatro programas de exemplo do guia entram sem alteração, como prova de compatibilidade com a sintaxe concreta original.

- Programas de erro ficam em pasta separada e declaram a fase em que devem falhar. Não basta falhar, precisa falhar na fase certa.
- Cobertura mínima combinada: condicional, repetição, atribuição, aninhamento, precedência, divisão com sinal, blocos vazios, entrada e cada extensão.

7. DIVISÃO DE TRABALHO

Responsável	Frente
Daniel Victor	Geração de código, modelo de saltos e rótulos, interface de linha de comando
Tiago Trindade	Análises léxica e sintática, precedência das comparações
Yann Fabber	Análise semântica, tabela de símbolos, mensagens de erro
Ralf Dwerich	Suíte de testes, extensões da seção Variações, documentação

8. SEQUÊNCIA DE EXECUÇÃO

Etapa 1	gramatica fechada, tokens definidos, esqueleto do projeto
Etapa 2	lexico e sintatico completos, acao ast conferindo as arvores
Etapa 3	semantico e interpretador de referencia, ja com casos de erro
Etapa 4	geracao de codigo, primeiro expressoes, depois if e while
Etapa 5	extensoes, suite completa de testes, README e este relatorio

Cada etapa só é considerada fechada quando os testes acumulados até ali passam. A ordem coloca o interpretador antes da geração de código de propósito: quando o assembly começar a sair errado, já existe um resultado correto para comparar.

9. RISCOS E MITIGAÇÃO

Risco	Mitigação
Rótulos repetidos em construções aninhadas	Contador global no gerador e teste com laço dentro de laço com condicional no meio
Ordem errada dos operandos em <code>CMP</code> , invertendo <code><</code> e <code>></code>	Teste dedicado somando um bit distinto por comparação, o que revela qual delas quebrou
Divisão com sinal sem estender para RDX	<code>cqto</code> antes de <code>idivq</code> e teste com dividendo negativo
Resultado maior que 255 perdido no código de saída do processo	Imprimir o resultado com <code>printf</code> e testar com 10 fatorial
Corretor sem máquina x86_64	O runner detecta a plataforma, avisa e cai para o interpretador de referência

10. CRITÉRIOS DE ACEITAÇÃO

- Os quatro programas do guia compilam sem alteração e devolvem 8, 45, 2 e 6.
- Toda a suíte passa nas duas vias de execução, interpretador e binário nativo.
- Todo caso de erro falha na fase declarada, com linha e coluna na mensagem.
- README explica como executar o compilador e os testes e lista as diferenças em relação ao guia, exigência da seção 8 da atividade.
- Nenhuma dependência externa além de Python 3 e gcc.